

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE
DELL'INFORMAZIONE

CORSO DI LAUREA TRIENNALE IN INFORMATICA

A FANCY L^AT_EX TEMPLATE FOR YOUR THESIS
AT UNINA

Relatori

Megadir. Prof. Guidobaldo Maria

RICCARDELLI

Duca Conte Francesco BARAMBANI

Baronessa Filiguelli DE BONCHAMPS

Correlatore

Megadir. Clam. Duca Conte Pier

Carlo ing. SEMENZARA

Candidato

Marco FESTA

N86004979

Anno Accademico 2025–2026

Sommario

Il presente lavoro di tesi illustra le attività di ricerca e sviluppo condotte durante il periodo di tirocinio presso **InfoCube**, focalizzandosi sull'estrazione, strutturazione e interrogazione semantica avanzata di documenti legali e normativi. Il progetto si inserisce nel contesto più ampio di **LUDOVICA** (*Language Understanding for Document Optimization and Validation In Civil Administration*), un motore generativo basato sull'Intelligenza Artificiale volto a supportare la Pubblica Amministrazione Locale nell'elaborazione e stesura di atti amministrativi.

Il problema tecnico principale affrontato in questo lavoro è la trasformazione di documenti PDF complessi in una base di conoscenza solida e interrogabile dalle macchine (**Knowledge Graph**). A tale scopo, la rigida gerarchia tipica degli atti normativi (Titoli, Capi, Articoli, Commi) è stata modellata e salvata all'interno del database a grafo **Neo4j**[1], permettendo di preservare fedelmente le relazioni strutturali e concettuali dei testi originali.

Per abilitare il recupero intelligente e contestuale delle informazioni, è stato implementato un sistema di **Semantic Search**. Questa fase ha richiesto il testing di molteplici **modelli di embedding**, allo scopo di individuare la soluzione più performante per la vettorializzazione del complesso gergo giuridico italiano. Al fine di validare le scelte architetturali, è stata condotta un'analisi comparativa diretta tra le performance di un approccio di ricerca vettoriale classica, basata sulla *cosine similarity*, e i vantaggi offerti dalla navigazione topologica e strutturale nel database a grafo.

SCRIVERE QUI I RISULTATI OTTENUTI, LE CONCLUSIONI E EVENTUALI PROSPETTIVE FUTURE.

Indice

1	Introduzione	1
1.1	Definizione del problema	1
1.2	Obiettivi del lavoro di tesi	2
1.3	Descrizione dell’approccio	2
1.4	Confronto con tecniche esistenti	2
1.5	Struttura della tesi	3
2	Tecnologie utilizzate	4
2.1	Linguaggi di Programmazione	4
2.2	Strumenti per l’estrazione del Testo	4
2.3	Scelta dei Database	6
2.4	LLM e Modelli di Embedding	6
2.5	Deployment e Containerizzazione	8
3	Architettura del Sistema	9
4	Implementazione Software	11
4.1	Estrazione dei Documenti	11
4.1.1	Open Data Loader e la Gestione dei Tagged PDF	11
4.1.2	Docling e l’Integrazione dei Motori OCR	12
4.1.3	Progettazione del Custom Triage Processor	13
4.2	Strutturazione dei Documenti tramite LLM	15
4.2.1	Il Modello e la Strategia di Generazione	15
4.2.2	Generazione della Struttura Gerarchica	15
4.2.3	Estrazione delle Relazioni Semantiche	16
4.3	Importazione dati nei Database	17
4.3.1	I Modelli di Dominio	17
4.3.2	Disaccoppiamento della Persistenza: Interfacce e Polimorfismo	18
4.3.3	Modalità di Accesso ai Dati: Il Repository Pattern	19
4.3.4	Dettagli Implementativi dell’Ingestion su Neo4j	19
4.3.5	Risultati e Metriche (DA AGGIORNARE)	20
4.4	Creazione degli Embedding	20

4.4.1	Differenza nell'Arricchimento Contestuale: Standard vs Late Chunks	21
Bibliografia		23

Elenco delle figure

3.1	Architettura del sistema	10
4.1	Flowchart del Custom Triage Processor	14
4.2	Class Diagram dei modelli di dominio	18

Elenco delle tabelle

-1-

Introduzione

CONTENTS: 1.1 Definizione del problema. 1.2 Obiettivi del lavoro di tesi. 1.3 Descrizione dell'approccio. 1.4 Confronto con tecniche esistenti. 1.5 Struttura della tesi.

1.1 Definizione del problema

La pubblica amministrazione locale italiana fa quotidianamente affidamento su una vasta e complessa mole di testi normativi, atti e delibere. L'elaborazione automatica di tali documenti è tuttavia ostacolata da diverse sfide intrinseche: i testi sono spesso archiviati in formati non strutturati (come i PDF), presentano difformità di impaginazione, impiegano un linguaggio burocratico peculiare e si articolano in una rigida e profonda struttura gerarchica. Quando si tenta di automatizzare la ricerca e la stesura di nuovi atti attraverso sistemi basati su Intelligenza Artificiale Generativa, l'approccio classico - basato unicamente sul recupero esplorativo di porzioni di testo destrutturato (frammenti di testo o "chunk") - si rivela insufficiente, portando inevitabilmente alla potenziale perdita del contesto legale e allo smarrimento delle relazioni gerarchiche originarie dell'atto documentale.

È in questo scenario che si inserisce il lavoro di tesi, svolto nell'ambito delle attività di tirocinio presso **InfoCube** per contribuire allo sviluppo del progetto **LUDOVICA**, il cui obiettivo finale è fornire ai funzionari della PA un motore generativo affidabile, capace di recuperare gli atti pregressi pertinenti e generare template documentali pronti all'uso. Affinché le risposte del sistema siano rigorose e verificabili è emersa l'esigenza di adottare un'architettura dati che introduca una solida topologia informativa.

Il problema tecnico fondamentale affrontato consiste quindi nella necessità di **convertire i documenti normativi complessi in una base di conoscenza interrogabile** (*Knowledge Graph*) che conservi fedelmente la struttura semantica e gerarchica originale (Titoli, Capi, Articoli).

1.2 Obiettivi del lavoro di tesi

Al fine di superare i limiti esposti e dotare il sistema di solidi meccanismi per l'interpretazione del linguaggio giuridico, il lavoro si è articolato nei seguenti obiettivi:

1. **Modellazione topologica e digitalizzazione:** Costruire una pipeline di inserimento per tradurre documenti legali non strutturati in un database a grafo (*Neo4j*), definendo rigorosamente la mappatura delle relazioni che legano tra loro le sezioni della gerarchia normativa.
2. **Semantic Search per il dominio giuridico:** Sviluppare il modulo di interrogazione intelligente del database e quantificare, mediante il testing di molteplici modelli di embedding, quale sia la soluzione più performante nel catturare la semantica intrinseca del linguaggio legale in lingua italiana.
3. **Analisi prestazionale degli approcci di Retrieval:** Eseguire un **confronto diretto** per misurare la differenza qualitativa delle risposte e confrontare le prestazioni fornite da un approccio di ricerca puramente vettoriale, basato sulla *cosine similarity*, rispetto a un'interrogazione che sfrutti attivamente la navigazione topologica sul grafo.

1.3 Descrizione dell'approccio

Per raggiungere gli obiettivi prefissati, è stata ideata un'architettura di elaborazione in grado di trasformare documenti non strutturati in una rete di nodi interconnessi. L'approccio proposto si articola in una pipeline a più fasi: inizialmente, i documenti originali (principalmente in formato PDF) vengono processati tramite strumenti di parsing avanzati e OCR per estrarne accuratamente il testo e convertirlo in un formato intermedio (Markdown). Successivamente, un *Large Language Model* analizza tale testo per riconoscerne la sintassi e la suddivisione gerarchica (Titoli, Capi, Articoli, Commi). Questi elementi vengono strutturati in formato JSON e iniettati all'interno del grafo, in cui ogni entità diventa un nodo e l'appartenenza a una data sezione diventa una relazione. Successivamente, i contenuti testuali vengono vettorializzati e salvati sotto forma di *embedding*. In fase di interrogazione, il sistema non si limita a cercare la similarità matematica tra le frasi, ma naviga esplicitamente la topologia del grafo per ricostruire l'intero iter contestuale, fornendo al modello generativo un prompt documentato, ricco e strutturalmente coerente.

1.4 Confronto con tecniche esistenti

Nel panorama attuale dell'Intelligenza Artificiale applicata all'elaborazione dei documenti, le tecniche di *Information Retrieval* più diffuse si basano principalmente su *Retrieval-Augmented Generation* (RAG) di tipo vettoriale. Quest'ultima prevede tipicamente la

suddivisione del documento in frammenti testuali di lunghezza fissa (*chunking*), la loro vettorializzazione e il successivo recupero basato sulla distanza semantica rispetto alla query dell'utente.

Tuttavia, queste tecniche presentano lacune significative quando applicate al dominio giuridico della pubblica amministrazione. Un approccio RAG standard, estraendo un singolo *chunk* isolato, perde inevitabilmente il perimetro normativo: ad esempio, potrebbe recuperare il comma di una legge senza specificare a quale Capo appartenga oppure omettendo dettagli cruciali come eventuali eccezioni specificate in altri paragrafi. L'approccio ibrido basato su *Knowledge Graph* (spesso definito *GraphRAG*) si rende pertanto strettamente **necessario**. Modellando esplicitamente le relazioni tra i componenti di una legge, esso supera i limiti del chunking piatto, consentendo di restituire un contesto normativo integro e affidabile. Questo aspetto è fondamentale per abbattere drasticamente le allucinazioni tipiche degli LLM e garantire l'assoluta affidabilità formale e legale delle risposte, un requisito imprescindibile per un software destinato agli operatori della PA.

1.5 Struttura della tesi

Il presente elaborato è organizzato in cinque capitoli, di seguito brevemente introdotti.

Il **Capitolo 2 — Tecnologie utilizzate** fornisce una panoramica dello stack tecnologico adottato: dai linguaggi di programmazione agli strumenti di parsing e OCR per l'estrazione del testo, passando per il database a grafo (Neo4j), fino ai modelli di linguaggio e di embedding impiegati per la strutturazione semantica e la vettorializzazione dei contenuti normativi.

Il **Capitolo 3 — Architettura del Sistema** illustra l'architettura complessiva del sistema, descrivendo i componenti principali e il flusso di dati attraverso le diverse fasi della pipeline: dall'acquisizione dei documenti grezzi fino all'interrogazione da parte degli utenti finali.

Il **Capitolo 4 — Implementazione Software** approfondisce nel dettaglio le singole fasi della pipeline di elaborazione: l'estrazione del testo dai documenti PDF, la strutturazione gerarchica operata dai Large Language Models, l'ingestion dei dati strutturati su Neo4j e la creazione degli embedding per la ricerca semantica.

Il **Capitolo 5 — Ricerca Semantica Vettoriale** presenta il setup sperimentale, le metriche di valutazione adottate e i risultati ottenuti con l'approccio di ricerca puramente vettoriale basato sulla *cosine similarity*.

Il **Capitolo 6 — Ricerca Semantica con Grafo** analizza i risultati del confronto tra l'approccio basato sul grafo e quello puramente vettoriale, evidenziando le differenze in termini di performance ed efficacia nella restituzione del contesto normativo.

–2–

Tecnologie utilizzate

Per far fronte alle sfide di estrazione, elaborazione, archiviazione e interrogazione dei documenti normativi, è stato impiegato uno stack tecnologico che combina strumenti di parsing avanzato, modelli di LLM ed embedding e database specializzati. Di seguito vengono presentate le tecnologie adottate, illustrando per ciascuna le motivazioni che ne hanno guidato la scelta.

2.1 Linguaggi di Programmazione

- **Python**[2]: È stato il linguaggio cardine per la costruzione dell'intera pipeline del progetto. La scelta è ricaduta su Python per diverse ragioni:
 - **Ecosistema dedicato ai modelli linguistici:** Python dispone di librerie mature e ampiamente supportate come `sentence-transformers` e `huggingface-hub`, che hanno permesso di integrare agevolmente i modelli di embedding e i workflow di Retrieval-Augmented Generation senza dover reimplementare componenti di basso livello.
 - **Integrazione con i database:** Il driver ufficiale per Neo4j offre API Python native, ben documentate e performanti, eliminando la necessità di adattatori o wrapper di terze parti.

2.2 Strumenti per l'estrazione del Testo

La fase di estrazione ha dovuto confrontarsi con documenti PDF eterogenei: alcuni nativamente digitali e ben strutturati, altri derivanti da scansioni cartacee di qualità variabile. Per questo motivo è stata adottata una combinazione di strumenti complementari.

- **Open Data Loader**[3]: Libreria open-source per l'estrazione e il caricamento di dati da fonti eterogenee. È stata selezionata per tre caratteristiche fondamentali:

- **Determinismo:** A differenza di approcci basati su modelli probabilistici, Open Data Loader estrae il testo in modo deterministico, garantendo riproducibilità e coerenza dei risultati — un requisito indispensabile nel dominio legale, dove ogni modifica del testo estratto tra un'esecuzione e l'altra minerebbe la fiducia nel sistema.
- **Conversione diretta in Markdown:** Il Markdown si rivela un formato testuale ottimale per i LLM. La sua sintassi leggera preserva fedelmente la struttura logica del documento originale, fornendo al modello un contesto già strutturato che riduce significativamente gli errori nella successiva fase di parsing gerarchico.
- **Docling**[4]: Libreria specializzata nell'analisi del layout documentale e nell'estrazione di contenuti strutturati. È stata integrata come supporto complementare a Open Data Loader per la gestione di PDF più complessi: supporta documenti multi-colonna, tabelle e layout non lineari, frequenti negli atti della pubblica amministrazione.
- **RapidOCR**[5]: Motore di *Optical Character Recognition* impiegato per digitalizzare i documenti derivanti da scansioni visive. La scelta è stata guidata da un'analisi comparativa tra diversi motori OCR:
 1. **EasyOCR:** Configurato di default all'interno di Docling, adotta modelli basati su Deep Learning altamente accurati ma estremamente esigenti in termini computazionali. Anche isolando l'elaborazione sulla singola pagina, il consumo di RAM superava costantemente la soglia critica dei 4 GB, rendendone impossibile l'utilizzo nell'ambiente di produzione containerizzato.
 2. **Tesseract OCR:** Si è dimostrato estremamente leggero e pienamente compatibile con i limiti di memoria imposti, scongiurando qualsiasi crash del container. Tuttavia, nel contesto specifico del linguaggio legale italiano, il motore non è stato in grado di interpretare correttamente il layout, fallendo l'estrazione del testo e restituendo output incompleti o disallineati rispetto alla struttura originale del documento.
 3. **RapidOCR:** Questo specifico modulo è stato infine selezionato per l'estrazione del testo dalle scansioni visive. RapidOCR si è rivelato il compromesso ingegneristico ideale, offrendo un perfetto bilanciamento tra l'accuratezza del riconoscimento del testo italiano e il consumo di RAM del container, che si è mantenuto costantemente sotto la soglia di guardia. A fronte di un tempo di computazione non trascurabile — quantificabile in circa 18-20 secondi per singola pagina scansionata — questo motore assicura la totale stabilità infrastrutturale dell'intera pipeline.

2.3 Scelta dei Database

La duplice natura del problema — da un lato la necessità di rappresentare relazioni gerarchiche complesse, dall'altro l'esigenza di una ricerca semantica efficiente — ha portato all'adozione di un database a grafo specializzato, affiancato dallo studio di metodologie di ricerca vettoriale classica basata su *cosine similarity*.

- **Neo4j**: Il database a grafo scelto come storage principale per la base di conoscenza normativa. La decisione di adottare un database a grafo anziché un tradizionale database relazionale è motivata da considerazioni sia modellistiche che prestazionali:
 - **Modellazione naturale della gerarchia normativa**: La struttura di un atto normativo — articolata in Titoli, Capi, Articoli e Commi — è intrinsecamente un grafo, in cui ogni elemento è un nodo e le relazioni di appartenenza sono archi. Neo4j consente di mappare questa topologia in modo diretto, senza le complessità delle JOIN ricorsive tipiche dei database relazionali.
 - **Flessibilità dello schema**: Il modello a grafo non impone uno schema rigido, permettendo di arricchire progressivamente la base di conoscenza con nuove tipologie di nodi e relazioni senza dover migrare lo schema esistente.
 - **Maturità e affidabilità**: Neo4j è il database a grafo più diffuso e maturo sul mercato, con una documentazione estesa e driver ufficiali per Python. Questo ha ridotto i rischi legati all'adozione di una tecnologia di nicchia in un contesto progettuale reale.
- **Cypher**[6]: Il linguaggio di interrogazione dichiarativo nativo di Neo4j, utilizzato per effettuare le query di inserimento e di ricerca topologica nel grafo.

2.4 LLM e Modelli di Embedding

Il cuore intelligente del sistema risiede nei modelli di linguaggio e di embedding, responsabili rispettivamente della strutturazione semantica dei documenti e della loro vettorializzazione per la ricerca.

- **Gemini 3.5 Flash**[7]: *Large Language Model* di Google, utilizzato tramite API per il compito cruciale della strutturazione sintattica dei documenti normativi. Le motivazioni alla base di questa scelta sono:
 - **Aderenza rigorosa allo schema JSON**: Gemini 3.5 Flash ha dimostrato un'eccellente capacità di produrre output JSON conformi a uno schema predefinito, un requisito fondamentale per garantire che la gerarchia normativa (Titoli, Capi, Articoli, Commi) fosse rappresentata in modo consistente e automaticamente processabile dalle fasi successive della pipeline.

- **Comprensione della lingua italiana:** Il modello ha mostrato una solida competenza nell’elaborazione del linguaggio burocratico italiano, riconoscendo correttamente la terminologia giuridica e le espressioni formulari tipiche degli atti amministrativi.
- **Rapporto costo/qualità:** Rispetto a modelli più potenti ma significativamente più costosi (come GPT-4 o Claude), Gemini 3.5 Flash ha offerto un compromesso ottimale tra qualità dell’output strutturato e costo per token, aspetto rilevante considerando l’elaborazione batch di centinaia di documenti.
- **Modelli di Embedding:** La vettorializzazione del testo normativo rappresenta un elemento critico per la qualità della ricerca semantica. Sono stati testati molteplici modelli per identificare la soluzione più efficace nel dominio giuridico italiano. I test più approfonditi hanno coinvolto i seguenti modelli:
 - **google/embeddinggemma-300m:** Modello sviluppato da Google che trasforma il testo in vettori a 768 dimensioni. È costruito sull’architettura Gemma 3 ed è addestrato su oltre 100 lingue. Una caratteristica distintiva è la possibilità di ridurre le dimensioni del vettore finale (a 512, 256 o 128) senza perdere qualità, rendendolo particolarmente efficiente su dispositivi con risorse limitate.
 - **perplexity-ai/pplx-embed-v1-0.6b:** Modello sviluppato da Perplexity AI che trasforma il testo in vettori a 1024 dimensioni, maggiori rispetto a Gemma per catturare più sfumature semantiche. È ottimizzato per applicazioni pratiche come ricerca semantica e sistemi RAG. Gli embeddings sono compressi in formato compatto (INT8), riducendo lo spazio di archiviazione necessario senza perdita significativa di qualità.
 - **perplexity-ai/pplx-embed-context-v1-0.6b:** Variante contestuale del modello Perplexity, anch’esso produce vettori a 1024 dimensioni. La differenza principale è che questo modello è addestrato per comprendere il *contesto* di un documento: quando processa i chunk, considera anche il resto del documento, non solo il frammento isolato. Questo lo rende più adatto per sistemi RAG dove il significato dei frammenti dipende dal contesto circostante. Questa tecnica è nota come *Late Chunking* e permette di ottenere rappresentazioni più ricche e accurate dei frammenti di testo.

La valutazione si è basata sulla capacità di ciascun modello di recuperare i commi pertinenti a partire da query in linguaggio naturale, misurata attraverso metriche standard di Information Retrieval quali *Precision@k*, e *Mean Reciprocal Rank* (MRR).

2.5 Deployment e Containerizzazione

- **Docker**[8]: L'intero ecosistema è stato modularizzato attraverso lo sviluppo di container isolati e specifici per ciascun componente della pipeline. Questa scelta architetturale è stata dettata da esigenze sia tecniche che operative:
 - **Isolamento delle dipendenze:** Ogni fase della pipeline (estrazione, strutturazione, ingestione, embedding) presenta dipendenze software potenzialmente conflittuali. Docker ha permesso di incapsulare ciascuna in un ambiente dedicato, eliminando i conflitti di versione tra librerie e garantendo la riproducibilità dell'ambiente di esecuzione su qualsiasi macchina.
 - **Controllo delle risorse:** I task più onerosi, in particolare l'OCR con RapidOCR e la generazione degli embedding, sono soggetti a picchi di consumo di CPU e RAM. Docker consente di imporre limiti precisi sulle risorse allocate a ciascun container, prevenendo fenomeni di *resource starvation* che potrebbero degradare le prestazioni dell'intero sistema.
 - **Modularità e manutenibilità:** La separazione in container indipendenti ha permesso di sviluppare, testare e aggiornare ciascun modulo della pipeline in isolamento, semplificando la manutenzione e facilitando l'onboarding di nuovi sviluppatori nel progetto LUDOVICA.
 - **Scalabilità orizzontale:** L'architettura a container costituisce un prerequisito naturale per l'orchestrazione tramite Kubernetes o Docker Swarm, aprendo la strada a futuri scenari di deployment distribuito in cui diverse istanze della pipeline possano operare concorrentemente su lotti distinti di documenti.

–3–

Architettura del Sistema

In questo capitolo viene presentata l'architettura complessiva del sistema sviluppato, illustrando i componenti principali e il flusso di dati attraverso le diverse fasi del processo. L'obiettivo è fornire una panoramica chiara e dettagliata delle scelte progettuali adottate per affrontare le sfide tecniche del progetto, tracciando il percorso logico che i dati seguono dal momento in cui vengono acquisiti fino alla fase di interrogazione e utilizzo da parte degli utenti finali.

La Figura 3.1 fornisce un diagramma riassuntivo di tale architettura, evidenziando i moduli principali e le interazioni tra di essi. Come illustrato nello schema, il sistema è concepito come una pipeline sequenziale.

Nella prima fase, vengono applicate tecniche specifiche per il recupero dei testi grezzi dai documenti normativi, con particolare attenzione alle librerie impiegate per il parsing e alle strategie di pulizia dei dati, fondamentali per mitigare rumore e inconsistenze strutturali.

Una volta estratto, il testo necessita di essere organizzato secondo le logiche giuridiche del documento. A tale scopo, la fase di strutturazione si avvale di Large Language Models (LLM) per la trasformazione del testo grezzo in un formato analizzabile. Il fulcro di questo passaggio è la definizione e l'utilizzo di uno schema JSON rigoroso, progettato appositamente per rappresentare e mantenere intatta la complessa gerarchia degli atti normativi, come la suddivisione in Titoli, Capi, Articoli e Commi.

I dati validati e strutturati vengono successivamente indirizzati verso i sistemi di archiviazione. Come mostrato in Figura 3.1, l'architettura prevede l'impiego del database a grafo (Neo4j), strutturato per abilitare un confronto architetturale e prestazionale tra due diversi paradigmi di ricerca:

- Il **database a grafo (Neo4j)** è deputato alla modellazione e memorizzazione delle informazioni strutturali e semantiche. Qui, i dati vengono inseriti prestando particolare attenzione alla rappresentazione dei metadati e delle relazioni gerarchiche e sequenziali che intercorrono tra le varie parti del documento normativo. Inoltre, il database ospita anche le rappresentazioni vettoriali del testo (embedding) per abilitare la ricerca semantica combinata alla topologia del grafo.

- L'implementazione di una **ricerca vettoriale classica** basata puramente sulla *cosine similarity*. Sfruttando le medesime rappresentazioni vettoriali archiviate, questo approccio è mirato a simulare la ricerca per similarità in un ambiente vettoriale puro, fungendo così da termine di paragone fondamentale per la successiva fase di benchmarking tra l'interrogazione puramente semantica e quella arricchita dalla struttura a grafo.

L'infrastruttura di storage fa da fondamento al modulo di implementazione della ricerca semantica. Questo componente descrive l'approccio adottato per permettere agli utenti di interrogare la base di conoscenza, includendo le motivazioni dietro la scelta di specifici modelli di embedding e la definizione delle metriche utilizzate per valutarne l'accuratezza.

Infine, per validare le scelte architetturali proposte, il sistema include un processo di benchmarking. Questo confronto analitico è stato condotto per misurare le prestazioni di ricerca, evidenziando le differenze in termini di performance ed efficacia tra l'approccio basato sulla navigazione strutturale nel grafo e quello puramente vettoriale classico basato sulla *cosine similarity*.

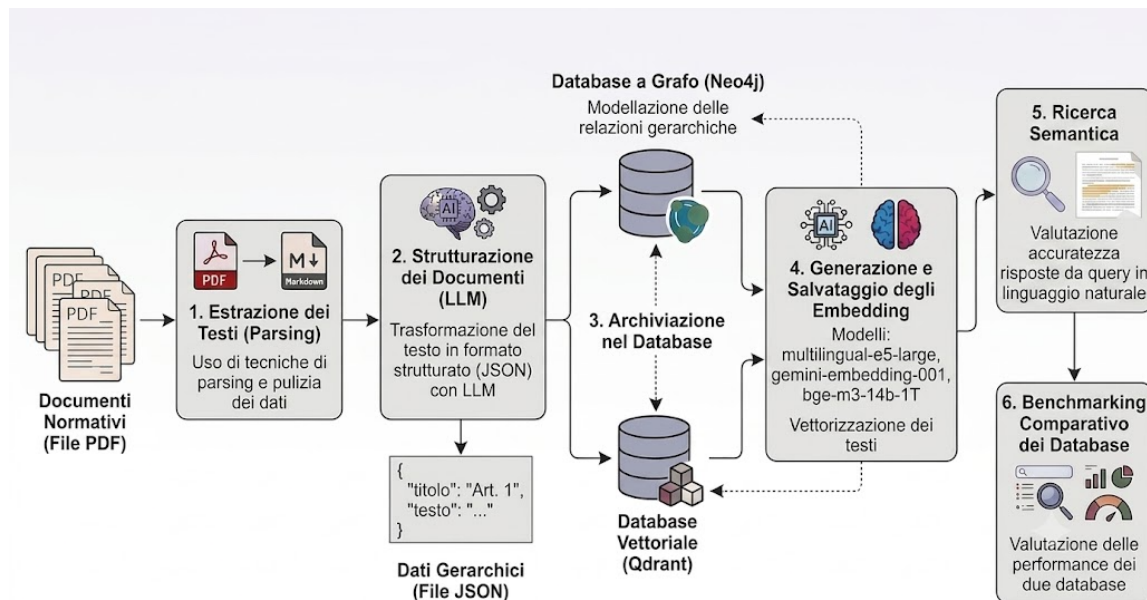


Figura 3.1: Architettura del sistema

–4–

Implementazione Software

CONTENTS: 4.1 Estrazione dei Documenti. 4.2 Strutturazione dei Documenti tramite LLM. 4.3 Importazione dati nei Database. 4.4 Creazione degli Embedding.

In questo capitolo verranno analizzate nel dettaglio le singole fasi della pipeline di elaborazione, partendo dalle strategie di estrazione del testo grezzo, passando per la strutturazione semantica operata dai Large Language Models, fino ad arrivare alle logiche di ingestione e interrogazione sui database

4.1 Estrazione dei Documenti

La fase iniziale della pipeline prevede l'estrazione del testo dai documenti normativi in formato PDF.

Nelle prime fasi di sviluppo del progetto, l'intero documento PDF veniva passato direttamente al Large Language Model (Gemini), demandando a quest'ultimo sia il compito di estrazione del testo grezzo sia quello di strutturazione sintattica. Questo approccio presentava forti criticità, esponeva il sistema al rischio di allucinazioni ed era inefficiente nel parsing di strutture complesse. Per superare tali limitazioni, la pipeline è stata modificata separando nettamente l'estrazione del testo dalla sua successiva strutturazione.

4.1.1 Open Data Loader e la Gestione dei Tagged PDF

Per l'estrazione del testo da documenti digitali nativi è stata adottata la libreria open-source *Open Data Loader*. Questo strumento converte localmente i file PDF in formati ottimizzati per i LLM, specificamente Markdown e JSON. Il vantaggio di questa soluzione risiede nella sua natura deterministica: a parità di input viene garantito il medesimo output, azzerando le variabilità tipiche dei modelli generativi.

Durante l'analisi dei documenti legali, una delle criticità maggiori riguarda i ritorni a capo spuri, generati per ragioni puramente stilistiche (ad esempio, il raggiungimento del

marginale della pagina). Se non gestiti correttamente, questi si traducono in doppi ritorni a capo nel testo Markdown finale, frammentando artificialmente i periodi e compromettendo la qualità della successiva fase di chunking. Per risolvere questa criticità alla radice, l'estrazione è stata configurata abilitando il parametro specifico `use_struct_tree=True`.

Quando questo parametro è attivo, la libreria modifica il suo approccio di lettura, verificando innanzitutto se il file analizzato appartiene alla categoria dei *Tagged PDF* [9]. A differenza dei PDF standard, che definiscono esclusivamente il layout bidimensionale e visivo, i Tagged PDF incorporano un livello gerarchico logico e invisibile: il *Logical Structure Tree*. Questo albero organizza il contenuto tramite elementi strutturali del tutto simili ai tag HTML (come `<H1>`, `<P>`, `<Table>`, `<L>`).

Di conseguenza, abilitando `use_struct_tree=True`, *Open Data Loader* è in grado di ignorare le fallaci euristiche geometriche e navigare direttamente la struttura logica del documento. Questo meccanismo permette all'estrattore di mappare ogni porzione di testo discriminando con precisione assoluta i semplici a capo visivi dalle reali interruzioni di paragrafo, restituendo un flusso di testo continuo e fedele all'originale.

Nel contesto di questa tesi, focalizzata sugli atti della Pubblica Amministrazione Locale, questa scelta tecnica si è rivelata non solo efficace, ma strategica. Le istituzioni pubbliche sono infatti vincolate da stringenti obblighi normativi sull'accessibilità digitale, che impongono la piena fruibilità dei documenti tramite tecnologie assistive, come gli screen reader. Per rispettare tali standard legali, i documenti pubblicati devono obbligatoriamente contenere un *Logical Structure Tree* rigoroso e ben formato. Pertanto, quello che per l'ente pubblico rappresenta un adempimento normativo inderogabile, si trasforma per la nostra pipeline di estrazione in una preziosa risorsa: una fonte di dati pre-strutturata in cui le gerarchie e l'ordine di lettura sono già nativamente validati alla radice.

4.1.2 Docling e l'Integrazione dei Motori OCR

L'approccio standard basato su *Open Data Loader* mostra evidenti limiti in presenza di documenti non strutturati, privi di testo selezionabile o derivanti da scansioni visive, in quanto tale libreria non integra nativamente alcun modulo OCR. Per sopperire a questa mancanza e gestire layout documentali complessi, è stata introdotta nel sistema la libreria *Docling*.

A differenza di *Open Data Loader*, *Docling* supporta l'integrazione diretta di motori OCR, configurandosi come un sistema avanzato di parsing e analisi del layout. Questo strumento, specializzato nell'estrazione di metadati, tabelle e testo da formati eterogenei, è stato quindi affiancato a *Open Data Loader* all'interno della pipeline, intervenendo come supporto specifico per processare tutti quei documenti in cui la sola estrazione strutturale risulta impossibile o inefficace.

L'introduzione dell'OCR ha sollevato complessità non trascurabili relative alla gestione delle risorse hardware all'interno dell'architettura a container. Il container Docker deputato all'estrazione operava infatti con un vincolo stringente di memoria RAM pari a un massimo di 4 GB, per questioni di risorse hardware. Durante i primi test, l'invio massivo di interi documenti scansionati causava la saturazione istantanea della memo-

ria, provocando l'intervento del modulo del kernel Linux (*Out-Of-Memory Killer*) che terminava forzatamente il container.

Per superare il problema del consumo di RAM e prevenire i crash di sistema, è stata implementata una strategia basata sull'elaborazione pagina-per-pagina. Come discusso nel Capitolo 2 relativo alle scelte tecnologiche, a seguito di un'analisi comparativa dei motori supportati da Docling, la scelta è ricaduta su **RapidOCR**. Questo motore ha permesso di mantenere il consumo di memoria costantemente sotto la soglia di guardia dei 4 GB, garantendo la totale stabilità infrastrutturale della pipeline senza sacrificare l'accuratezza del riconoscimento sul testo legale italiano.

4.1.3 Progettazione del Custom Triage Processor

Il processore di triage nativo di Open Data Loader prevede un flusso standard in cui ogni pagina viene analizzata e smistata: le pagine semplici vengono processate tramite il processo di Open Data Loader (una JVM), mentre le pagine complesse vengono delegate al backend basato su Docling.

Tuttavia, l'applicazione con l'uso del parametro `use_struct_tree=True` introduce un bug logico nel triage di default: i documenti non espressamente *tagged* vengono forzati all'interno del processo standard di Open Data Loader, non venendo mai instradati verso il backend di Docling, escludendo così l'analisi per pagine troppo complesse oppure l'attivazione dell'OCR dove necessario.

Per risolvere questa problematica è stato progettato e implementato un **Custom Triage Processor**. Come illustrato nel diagramma di flusso in **Figura 4.1**, il componente corregge le anomalie di instradamento agendo secondo una logica a tre fasi:

- **Fase 1: Verifica preliminare della struttura.** Come si evince dal primo blocco decisionale della Figura 4.1, non appena il file PDF viene acquisito, il sistema ne analizza i metadati per verificare se è codificato come *Tagged PDF*. In caso affermativo, l'intero documento viene elaborato direttamente da Open Data Loader sfruttando l'albero strutturale nativo. Questo approccio garantisce la massima velocità di esecuzione e una pulizia ottimale del testo estratto.
- **Fase 2: Frazionamento del documento e stima della complessità.** Se il documento risulta privo di struttura (*Non-Tagged*), il flusso prevede la divisione in singole pagine indipendenti. Questa separazione è fondamentale per prevenire sovraccarichi di memoria RAM durante le elaborazioni successive. Per ciascuna pagina, il sistema calcola poi la densità dei caratteri testuali per valutarne il grado di complessità.
- **Fase 3: Triage ed elaborazione mirata della pagina.** Sulla base dell'analisi precedente, ogni singola pagina subisce il processo di smistamento intelligente mostrato nella parte inferiore della Figura 4.1, venendo indirizzata allo strumento di estrazione più adeguato:

- Se la pagina presenta un layout lineare e un'elevata densità di testo nativo, viene classificata come *Semplice* e affidata al processo di Open Data Loader.
- Se la pagina contiene elementi geometrici articolati o tabelle annidate, ma il testo rimane comunque selezionabile, viene classificata come *Complessa* e processata tramite *Docling Standard* (senza abilitare l'OCR).
- Se la pagina è priva di testo nativo (es. una fotocopia), viene classificata come *Scansione* e inviata al container specializzato di *Docling*, dove interviene il motore *RapidOCR* per il riconoscimento ottico dei caratteri.

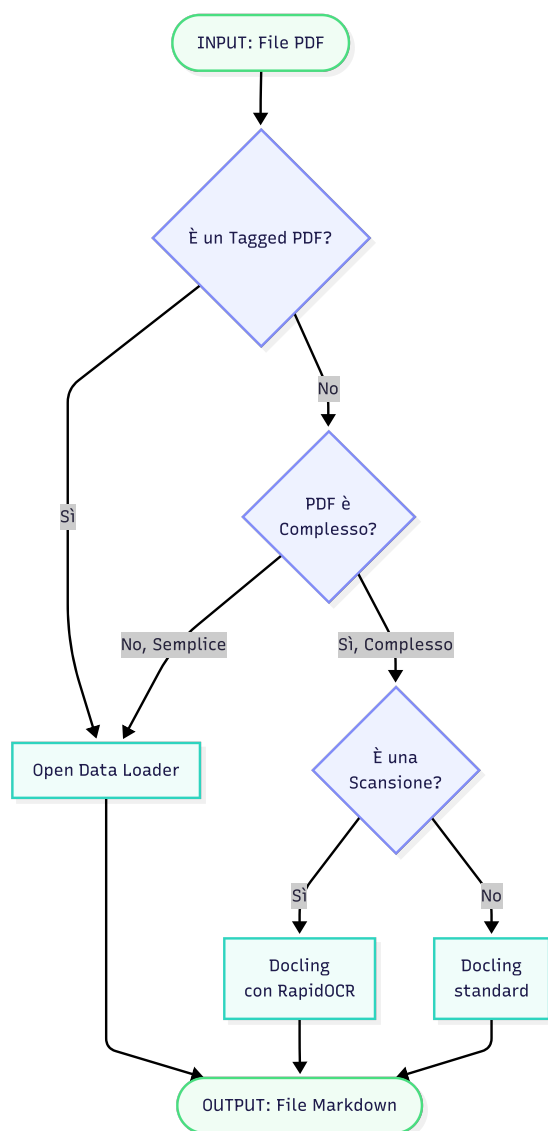


Figura 4.1: Flowchart del Custom Triage Processor

4.2 Strutturazione dei Documenti tramite LLM

Una volta ultimata la fase di estrazione del testo grezzo, illustrata nella sezione precedente, i documenti normativi si presentano sotto forma di file testuali piatti in formato Markdown. Tali file, pur contenendo l'intero testo originale, sono del tutto privi di metadati strutturali e di una formattazione gerarchica chiara. Poiché l'architettura di sistema prevede l'ingestione di questi documenti all'interno di un database a grafo (Neo4j) al fine di confrontare la navigazione topologica con la classica ricerca vettoriale basata su *cosine similarity*, risulta indispensabile trasformare il testo non strutturato in una gerarchia rigorosa. L'archiviazione del solo testo grezzo, infatti, risulterebbe del tutto inadeguata e priva di utilità pratica. Una strutturazione accurata rappresenta il prerequisito fondamentale per abilitare qualsiasi operazione avanzata di *information retrieval*, garantendo che i dati possano essere interrogati in modo efficiente e contestuale.

4.2.1 Il Modello e la Strategia di Generazione

Per affrontare il compito cruciale della strutturazione sintattica, è stato impiegato il Large Language Model Gemini 3.5 Flash. A livello di architettura software, il processo è diviso in due step distinti: la generazione di una struttura JSON gerarchica e l'estrazione delle relazioni semantiche tra le entità legali.

4.2.2 Generazione della Struttura Gerarchica

L'efficacia e l'accuratezza del processo di strutturazione si fondano su un *System Prompt* ingegnerizzato ad hoc. Il suo compito principale è quello di mappare il contenuto all'interno del seguente schema logico:

- **Metadati (metadata):** Il modello estrae e categorizza i dati identificativi dell'atto. Vengono isolati campi chiave come la data di stipula, il titolo completo e il nome del file originale.
- **Sezioni Introduttive e Conclusive:** Vengono separate dal corpo normativo tutte le sezioni di contorno. Tra queste figurano le premesse iniziali e le epigrafi (preface), gli indici (preamble), le dichiarazioni di chiusura con le relative firme (conclusions) e gli eventuali documenti annessi (attachments).
- **Corpo del Documento (body):** Rappresenta il nucleo dell'algoritmo di strutturazione. Il modello deve nidificare il testo seguendo fedelmente la gerarchia complessa e rigida tipica degli atti normativi italiani: **Titoli > Capi > Articoli > Commi**.

Per garantire la massima coerenza dei dati all'interno del Knowledge Graph, sono state imposte al LLM regole tassative di parsing:

- **Normalizzazione Numerica:** Tutti gli identificativi gerarchici originariamente espressi in numeri romani o in forma testuale (es. “TITOLO I”, “CAPO IV”) devono essere categoricamente convertiti in numeri interi arabi (1, 4, ecc.).
- **Gestione Dinamica dei Commi:** Poiché nei testi di legge i commi non sono sempre esplicitamente numerati, il modello è istruito a dedurli analizzando le interruzioni logiche dei paragrafi, assegnando loro una numerazione progressiva.
- **Ottimizzazione dello Schema:** Per mantenere leggeri i file JSON, il modello è programmato per non inizializzare chiavi con valori vuoti o nulli; se un livello gerarchico (come un Capo o un Titolo) non trova riscontro nel testo originale, la proprietà viene semplicemente omessa.

L'esecuzione di questo processo restituisce una directory di documenti JSON puliti e strutturati gerarchicamente, pronti per essere archiviati e per affrontare il successivo step: l'estrazione delle relazioni semantiche tra le entità legali.

4.2.3 Estrazione delle Relazioni Semantiche

La sola scomposizione del testo normativo in una struttura gerarchica ad albero rappresenta una condizione necessaria ma non sufficiente per la costruzione di un vero e proprio *Knowledge Graph*. Per sfruttare appieno la potenza topologica di un database a grafo come Neo4j, è indispensabile individuare e mappare esplicitamente i collegamenti e le dipendenze logiche che intercorrono tra le diverse disposizioni giuridiche all'interno dello stesso atto. A questo scopo, la pipeline prevede un secondo step logico interamente dedicato all'estrazione delle relazioni semantiche. In questa fase, l'input non è più il testo grezzo, ma i file JSON prodotti nello step precedente, che quindi agiscono come una base di conoscenza consolidata. Ciascun documento viene quindi analizzato dal LLM Gemini 3.5 Flash, guidato da un *System Prompt* specializzato. Tale prompt è stato progettato per riconoscere i costrutti linguistici del gergo giuridico che indicano un collegamento formale tra le parti del testo. Le relazioni estratte sono:

- **CITA:** Un articolo o comma fa riferimento a un'altra disposizione.
- **ABROGA:** Una disposizione annulla la validità giuridica di un'altra.
- **DEROGA:** Un elemento introduce un'eccezione a una regola generale.
- **SOSTITUISCE:** Una disposizione prende il posto di un'altra.
- **INTERPRETA:** La comprensione di una norma dipende dalla lettura di un'altra.

L'output di questa analisi è una rete di collegamenti formali, dove per ciascuna relazione vengono specificati il nodo di partenza, il nodo di arrivo e il tipo di legame. Queste informazioni sono esportate in un nuovo set di file JSON.

Questo approccio a due stadi permette di separare le responsabilità e ridurre il carico cognitivo del modello a ogni ciclo. In questo modo si produce un dataset pulito e affidabile, pronto per essere caricato nel grafo.

4.3 Importazione dati nei Database

Una volta completata la strutturazione dei documenti legali in formato JSON, si apre la fase di persistenza e indicizzazione dei dati. Nel contesto di questo progetto di tirocinio, l'architettura prevede il caricamento dei dati estratti all'interno del database a grafo Neo4j.

Per garantire la flessibilità nell'applicazione di queste logiche di indicizzazione e favorire un ambiente di sviluppo pulito e modulare, lo sviluppo del software ha visto l'applicazione del *Repository Pattern* e l'uso di interfacce polimorfiche.

4.3.1 I Modelli di Dominio

Prima di analizzare le strategie di persistenza, è fondamentale descrivere come i dati estratti vengano rappresentati in memoria. All'interno della cartella `models` di questo step, il sistema definisce le entità fondamentali del dominio normativo utilizzando il paradigma ad oggetti. Queste classi operano essenzialmente come *Entities*: strutture dati pure, prive di logica di persistenza diretta o dipendenze verso i DBMS, il cui unico scopo è incapsulare i dati e preservare i vincoli di integrità del dominio.

L'architettura dei modelli rispecchia fedelmente la scomposizione gerarchica del testo legale:

- **Entity**: Rappresenta l'ente pubblico emittente (ad esempio, un Ministero o un Comune), memorizzando i metadati identificativi dell'istituzione.
- **LegalAct**: Modella l'atto normativo nella sua interezza. Al suo interno, oltre ai metadati generali del documento, espone il metodo `from_json_file(file_path)`, delegato al parsing del file JSON e alla deserializzazione ricorsiva dei suoi elementi strutturali.
- **StructuralElement**: Una classe base polimorfica specializzata nelle sottoclassi `Title`, `Chapter` e `Article`. Permette di mappare i livelli intermedi di aggregazione del testo, mantenendo traccia del numero d'ordine e del titolo della sezione.
- **Paragraph**: Rappresenta il singolo comma contenente il testo, ovvero l'unità atomica d'informazione, sul quale verranno poi calcolati gli embedding.

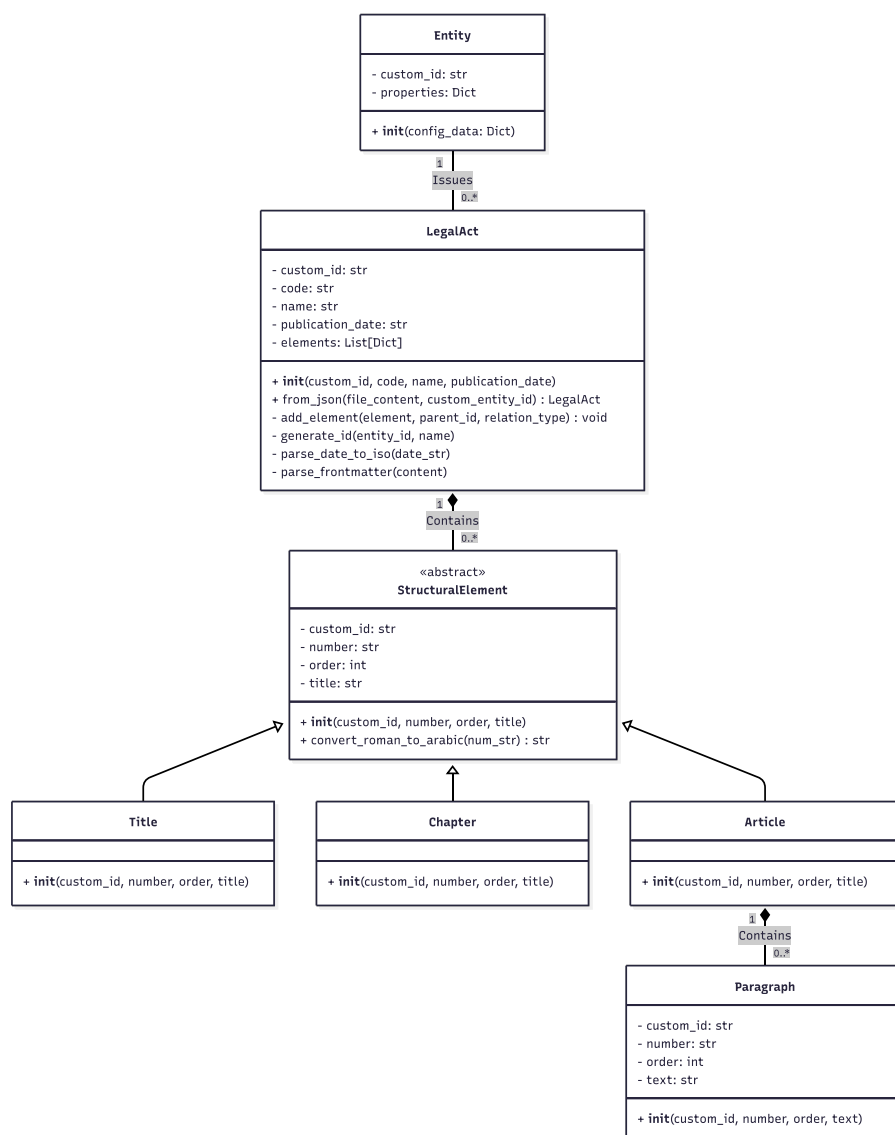


Figura 4.2: Class Diagram dei modelli di dominio

4.3.2 Disaccoppiamento della Persistenza: Interfacce e Polimorfismo

L'adozione di un'interfaccia comune di importazione nasce dalla precisa volontà di disaccoppiare la logica di orchestrazione e lettura dei dati dalla loro effettiva persistenza fisica. Sebbene l'attuale implementazione preveda l'utilizzo esclusivo del database a grafo Neo4j, questa scelta architetturale si rivela strategica per sviluppi futuri. Qualora si rendesse necessario integrare nuovi DBMS (ad esempio, un database puramente vettoriale dedicato esclusivamente agli embedding), sarà sufficiente implementare una nuova classe concreta senza dover alterare il flusso principale del programma. Inoltre, il disaccoppiamento

garantisce un'elevata testabilità del codice, permettendo di validare la logica di dominio e il parsing dei documenti simulando le operazioni di salvataggio indipendentemente dal database sottostante.

Nel sistema sviluppato, l'interfaccia astratta di riferimento è rappresentata dalla classe `DBImporterInterface`. Essa definisce tre metodi fondamentali che ogni classe concreta deve implementare:

- `connect()`: per la gestione e la validazione della connessione con l'istanza specifica del database;
- `import_legal_act(entity, legal_act)`: per l'esecuzione materiale della logica di Ingestion dell'atto normativo e dei metadati dell'ente emittente;
- `close()`: per la chiusura sicura delle sessioni.

Nello script principale `main.py`, il ciclo di esecuzione batch istanzia dinamicamente l'oggetto concreto, delegando a quest'ultimo i dettagli tecnici dell'inserimento. Questa architettura estendibile riduce a zero l'accoppiamento tra il livello applicativo e l'infrastruttura dati.

4.3.3 Modalità di Accesso ai Dati: Il Repository Pattern

Se l'interfaccia di importazione isola le strategie nella loro interezza rispetto all'orchestrazione principale, il *Repository Pattern* interviene specificamente all'interno dell'architettura di persistenza dei database per mediare tra lo strato dei modelli di dominio e lo strato di accesso ai dati fisici. L'obiettivo principale del Repository Pattern è centralizzare la logica di costruzione ed esecuzione delle query, offrendo al client un'interfaccia astratta per poter salvare i dati. In questo modo, l'importer concreto delega l'interazione alle singole repository, le quali ricevono l'oggetto di dominio da persistere, nascondendo completamente i dettagli delle implementazioni delle query.

4.3.4 Dettagli Implementativi dell'Ingestion su Neo4j

Nel contesto di Neo4j, un atto legale non può essere salvato tramite un'unica operazione atomica standard, a causa delle profonde interconnessioni tra i nodi della gerarchia. Aniché concentrare la scrittura di stringhe Cypher complesse all'interno della classe `Neo4jImporter`, la responsabilità è stata distribuita su una serie di repository specializzati, ognuno deputato alla gestione del ciclo di vita di un singolo modello di dominio:

- `EntityRepository`: Gestisce l'inserimento dei nodi relativi agli enti istituzionali.
- `LegalActRepository`: Inserisce il nodo radice dell'atto e lo ancora all'ente emittente.
- `StructuralElementRepository`: Incapsula la logica Cypher necessaria a creare i nodi intermedi del grafo e a tessere gli archi gerarchici verso i rispettivi nodi padri.

- **ParagraphRepository**: Si occupa esclusivamente della persistenza dei nodi foglia contenenti il testo dei commi.

Per quanto riguarda il salvataggio delle relazioni trasversali (ad esempio, le citazioni tra diversi atti normativi), queste informazioni vengono lette da file JSON esterni dedicati. Il metodo che si occupa di questa operazione attraverso i `custom_id` dei nodi salvati e indicizzati sul grafo trova in maniera quasi istantanea i nodi correlati dall'estratto JSON ed esegue una query Cypher dedicata procedendo alla creazione dell'arco semantico.

Listing 4.1 Esempio di relazione estratta in formato JSON

```

1: {
2:   "sourceId": "ente-provincia-taranto__CCNL-22-02-2010__article
      -3__paragraph-1",
3:   "relationshipType": "CITES",
4:   "targetId": "ente-provincia-taranto__CCNL-22-02-2010__article
      -2"
5: }
? INSERIRE ANCHE QUERY CYPHER ?

```

4.3.5 Risultati e Metriche (DA AGGIORNARE)

INSERIRE METRICHE DI PERFORMANCE E RISULTATI DELL'INGESTION, TEMPI DI ESECUZIONE, NUMERO DI NODI E RELAZIONI INSERITI.

4.4 Creazione degli Embedding

L'operazione di vettorizzazione, o di *embedding*, rappresenta un passaggio cruciale per abilitare le funzionalità di ricerca semantica all'interno del database a grafo. La pipeline supporta tre modelli di embedding, dettagliati in section 2.4:

- `google/embeddinggemma-300m`
- `perplexity-ai/pplx-embed-v1-0.6b`
- `perplexity-ai/pplx-embed-context-v1-0.6b`

I vettori vengono archiviati direttamente come proprietà all'interno dei nodi (`:Paragraph`) già esistenti nel grafo. Questo evita la necessità di un database vettoriale separato e consente al modello di ricerca di operare direttamente su Neo4j.

Neo4j, inoltre, supporta la creazione di indici vettoriali utilizzando algoritmi di ricerca approssimata (*Approximate Nearest Neighbor Search*, ANN). Invece di calcolare la distanza coseno tra il vettore di query e tutti gli altri vettori (operazione $O(n)$ molto costosa), questi indici riducono drasticamente lo spazio di ricerca. Di conseguenza, il tempo di esecuzione delle query passa da secondi a millisecondi, rendendo la ricerca praticabile anche su documenti normativi di grandi volumi.

4.4.1 Differenza nell'Arricchimento Contestuale: Standard vs Late Chunking

Il sistema implementa due approcci diversi per fornire *contesto* ai commi. Se si fornisce al modello solo la stringa del comma «È punito con la reclusione da 1 a 3 anni», l'embedding sarà di scarsa qualità perché manca il soggetto e l'argomento (definiti nell'articolo o titolo).

Ecco come le due metodologie risolvono il problema:

A. Arricchimento Standard (Modelli: Gemma 300m e PPLX-v1)

In questo approccio, il contesto viene iniettato testualmente attraverso *String Concatenation*. Per ciascun comma, viene recuperato il contesto gerarchico risalendo l'albero strutturale (articolo, capitolo, titolo, atto) tramite query Cypher. Tutte queste informazioni vengono concatenate in un'unica stringa, producendo un testo inviato al modello simile al seguente:

```
«Atto: Costituzione | Titolo 1 | Art. 13 | Comma 1 | La libertà  
personale è inviolabile.»
```

In questo approccio, ogni comma viene trattato come un *chunk isolato* al momento dell'embedding. Sebbene il modello riceva il testo arricchito dai metadati gerarchici, il contesto rimane limitato a questo preambolo testuale statico.

B. Arricchimento tramite Late Chunking (Modello: PPLX-Context)

Questo approccio impiega una vera e propria contestualizzazione a livello di rete neurale. A differenza dell'approccio standard, non utilizza preamboli testuali pre-calcolati, ma affida la comprensione semantica all'algoritmo di *self-attention* del modello.

Il sistema recupera dal database, tramite query Cypher, l'intero articolo con tutti i commi che lo compongono, mantenendo traccia dell'ordine sequenziale tra di essi. Invece di inviare al modello i commi singolarmente, il sistema li presenta come un'unica sequenza ordinata.

Il modello processa l'intero articolo permettendo ai token di ciascun comma di interagire semanticamente con i token dei commi adiacenti tramite il meccanismo di *self-attention*. Solo al termine di questo processo, il modello estrae e restituisce i singoli vettori per ciascun comma. Questa tecnica, nota come *late chunking*, cattura il significato globale dell'articolo pur mantenendo la precisione a livello di comma.

DA RISCRIVERE

Bibliografia

- [1] *Neo4j Graph Platform*. URL: <https://neo4j.com>.
- [2] *Python Language Reference*. URL: <https://www.python.org>.
- [3] *Open Data Loader*. URL: <https://github.com/opendataloader-project/opendataloader-pdf>.
- [4] *Docling: Document Parsing and Layout Analysis*. URL: <https://github.com/DS4SD/docling>.
- [5] *RapidOCR: OCR Engine*. URL: <https://github.com/RapidAI/RapidOCR>.
- [6] *Cypher Query Language Reference*. URL: <https://neo4j.com/docs/cypher-manual>.
- [7] *Gemini 3.5 Flash Model Card*. URL: <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-5-Flash-Model-Card.pdf>.
- [8] *Docker: Containerization Platform*. URL: <https://www.docker.com>.
- [9] *Document management — Portable document format — Part 1: PDF 1.7*. Standard. Sezione 14.8: Tagged PDF. Geneva, Switzerland: International Organization for Standardization, 2008. URL: <https://www.iso.org/standard/51502.html>.